

---

## TASKS TO COMPLETE

### Designing a class

When designing a class to model an entity, it's often best to start with what methods the class should have. The methods are the “ways of interacting with” the entity, so you should think about what you need the entity for in your software — how are you going to use it?

Once you have a list of methods, you can work out what fields the class needs. When designing the class this way, we only write fields if they're needed to make the methods work.

### Task 1 – User Account

Design a class to model a user account, such as on a website. Following Java's naming conventions, this class should be named either `User` or `UserAccount`.

In our software, we might need to “interact” with a user account in a few ways:

- Get the username.
- Check if a correct password was entered.
- Change the user's password.

This means our `User` class ought to have three methods, which could be named `getUsername`, `checkPassword` and `setPassword`.

You will also want a constructor which requires us to choose a username and password for the new account.

Note that we don't have a `setUsername` method since a username can't be changed after the account is registered. We definitely don't want a `getPassword` method, because it could compromise the security of the user's account; if we “interacted” with a user by asking for their password, then they *shouldn't* answer this question.

To make the methods work, the `User` class must have `username` and `password` fields, which are both strings. These will be private, to ensure the `User` objects are properly encapsulated; the `username` field will also be `final` since it shouldn't be changed.

You should also think more carefully about what the `setPassword` method does — to be secure, the user should only be able to change their own password. We can ensure this by requiring the old password as a parameter. The method will only change the user's password if the old password is entered correctly. We should also print a message to say whether the password was changed successfully or not.

Check the String `equals` method to determine if two Strings are equal:

<https://docs.oracle.com/javase/8/docs/api/java/lang/String.html#equals-java.lang.Object->

---

---

## Task 2 – Bank Account

Let's design a class to model a bank account. Following Java's naming conventions, this class should be named either `BankAccount` or just `Account`.

In our software, we might need to "interact" with a bank account in several ways:

- Get the account number.
- Get who owns the account.
- Get the current balance.
- Deposit money.
- Withdraw money, up to the current balance.

This means the `BankAccount` class ought to have five methods, which could be named `getAccountNumber`, `getAccountHolder`, `getBalance`, `deposit` and `withdraw`. We will also want a constructor which requires us to choose an account number and account-holder for the new bank account.

These "ways of interacting" with a bank account imply that the bank account must have some data associated with it — an account number, an account-holder, and a current balance. This means the `BankAccount` class ought to have three fields, which could be named `accountNumber`, `accountHolder` and `balance`. A bank account's number and account-holder can't be changed after the account is created, so these fields will be `final`.

Part of modelling is choosing data types:

- We will represent the account number as a `String`. A string is better than an `int` here because it can start with the digit 0.
- We will represent the account-holder as a `User`, not a `String`. This means our model is a bit more complicated overall since it involves two classes, but the advantage is that each class will be simpler. This way the `BankAccount` class doesn't depend on how passwords are verified or who is allowed to change them.
- We will represent the balance as a `double` because it has two decimal places (for the pennies). However, `double` values have limited precision, so small rounding errors can accumulate if many deposits and withdrawals are made.

The `deposit` and `withdraw` methods will need to check whether the amount deposited or withdrawn is valid. We should also print a message to say whether or not the action was successful.

Write the `BankAccount` class based on the above specification. The `User` class is given below.